DESIGN & ANALYSIS OF ALGORITHMS

QUESTION BANK • MODULES 1–5

MCA • Course Code: 24MCDAAT302
Separate question banks for all five modules, arranged by 4-mark and 8-mark questions, with priority labels and practice problems.

How to use this bank

- **Very High:** study first. These are core topics and/or directly supported by the uploaded October 2025 DAA paper.
- **High:** prepare after the very-high-priority topics.
- **Medium:** use for coverage and backup options.
- The labels are study priorities, not guaranteed exam questions or official topic-wise weightage.

Answer length requested

4 marks: approximately 2 handwritten pages. 8 marks: approximately 3–4 handwritten pages. For long answers, include definition, explanation, algorithm/pseudocode where applicable, worked example or diagram, complexity/result and conclusion.

This bank is based on the uploaded DAA module notes and the October 2025 previous question paper. Question wording is phrased for practice; it is not a claim that every question appeared verbatim in the paper.

MODULE 1 — Algorithm Analysis, Recurrences & Divide and Conquer

Scope: Algorithm analysis; time and space complexity; asymptotic notation; recurrence equations; substitution and iteration methods; divide and conquer; Merge Sort; Quick Sort; matrix multiplication.

> **Previous-paper signal:** The uploaded October 2025 paper includes Quick Sort worst-case $O(n^2)$ as a 4-mark question. Its 8-mark alternatives include asymptotic complexity with O/Ω/Θ and divide and conquer in Merge Sort.

A. 4-MARK QUESTIONS

Write each as a complete short answer, aiming for about 2 handwritten pages as requested. Include a definition, key points and a small example/diagram where appropriate.

[VERY HIGH] 1. Define algorithm analysis. Differentiate a priori and a posteriori analysis.

[VERY HIGH] 2. Define time complexity and space complexity. Explain the fixed and variable parts of space complexity.

[VERY HIGH] 3. Explain Big-O, Big-Ω and Big-Θ notations.

[VERY HIGH] 4. Quick Sort has worst-case time complexity $O(n^2)$. Justify this statement.

[HIGH] 5. Explain constant, linear, logarithmic and quadratic time complexities with examples.

[HIGH] 6. What is a recurrence relation? Explain the substitution method.

[HIGH] 7. Explain the iteration method for solving recurrence equations.

[MEDIUM] 8. Explain the divide-and-conquer strategy and name its main stages.

[MEDIUM] 9. Differentiate Merge Sort and Quick Sort.

[MEDIUM] 10. Explain the space-complexity expression S(p) = A + S■(I) as given in the notes.

B. 8-MARK QUESTIONS

Aim for 3–4 handwritten pages. Algorithm questions should include steps/pseudocode, a trace or diagram, and complexity where the notes support it.

[VERY HIGH] 1. What is asymptotic complexity of an algorithm? Explain the three notations used to represent asymptotic complexity.

[VERY HIGH] 2. Explain the divide-and-conquer approach used in Merge Sort with a suitable example.

[VERY HIGH] 3. Explain Quick Sort with an example. Discuss its best, average and worst-case behaviour.

[HIGH] 4. Solve a recurrence equation using the substitution method, showing each step.

[HIGH] 5. Solve a recurrence equation using the iteration method, showing the expansion and final result.

[HIGH] 6. Explain recurrence equations and the methods used to solve them, including substitution and iteration.

[HIGH] 7. Explain time and space complexity with suitable examples and common growth orders.

[MEDIUM] 8. Explain divide and conquer for matrix multiplication as covered in the module notes.

[MEDIUM] 9. Compare Merge Sort and Quick Sort in terms of working principle, complexity and memory requirements.

C. WORKED-PROBLEM / DIAGRAM PRACTICE

1. Trace Merge Sort on an input array of your choice, showing each division and merge.
2. Trace Quick Sort on a small array, identifying the pivot and partition at each stage.
3. Given a recurrence from class notes, solve it using substitution or iteration and show the algebra.

MODULE 2 — Greedy Strategy & Dynamic Programming

Scope: Greedy strategy and control abstraction; feasible and optimal solutions; minimum spanning tree; Kruskal's and Prim's algorithms; fractional knapsack; job sequencing with deadlines; dynamic programming; Travelling Salesperson Problem; shortest paths including Dijkstra and Bellman–Ford.

> **Previous-paper signal:** The uploaded October 2025 paper includes MST definition with example (4 marks), greedy strategy with control abstraction (8 marks), and TSP using dynamic programming (8 marks).

A. 4-MARK QUESTIONS

Write each as a complete short answer, aiming for about 2 handwritten pages as requested. Include a definition, key points and a small example/diagram where appropriate.

[VERY HIGH] 1. Define a minimum spanning tree and illustrate it with an example.

[VERY HIGH] 2. Define the greedy method. Explain feasible solution and optimal solution.

[HIGH] 3. List and explain the components of a greedy algorithm.

[HIGH] 4. Differentiate greedy method and dynamic programming.

[HIGH] 5. State the main steps of Kruskal's algorithm.

[HIGH] 6. State the main steps of Prim's algorithm.

[HIGH] 7. What is the fractional knapsack problem? Explain the greedy selection criterion.

[HIGH] 8. Define the Travelling Salesperson Problem (TSP).

[MEDIUM] 9. Explain job sequencing with deadlines.

[MEDIUM] 10. Differentiate Dijkstra's algorithm and Bellman–Ford algorithm.

B. 8-MARK QUESTIONS

Aim for 3–4 handwritten pages. Algorithm questions should include steps/pseudocode, a trace or diagram, and complexity where the notes support it.

[VERY HIGH] 1. Explain the greedy strategy with the control abstraction model.

[VERY HIGH] 2. Explain the Travelling Salesperson Problem using Dynamic Programming.

[HIGH] 3. Construct a minimum spanning tree using Kruskal's algorithm for a given weighted graph. Show the selected edges and total weight.

[HIGH] 4. Construct a minimum spanning tree using Prim's algorithm for a given weighted graph. Show the steps.

[HIGH] 5. Solve a fractional knapsack problem using the greedy method, showing value/weight ratios and total profit.

[HIGH] 6. Explain dynamic programming and compare it with the greedy method using examples from the notes.

[HIGH] 7. Explain a shortest-path algorithm using a worked graph example: Dijkstra or Bellman–Ford.

[MEDIUM] 8. Explain job sequencing with deadlines using a worked example.

[MEDIUM] 9. Explain the components and control abstraction of the greedy method, with a suitable application.

C. WORKED-PROBLEM / DIAGRAM PRACTICE

1. Use Kruskal's algorithm on a weighted graph: list edges in selection order, reject any edge that creates a cycle, and calculate total MST weight.
2. Use Prim's algorithm on a weighted graph and show how the tree grows from the starting vertex.
3. Solve a fractional knapsack example using value/weight ratios.
4. Practise a small TSP dynamic-programming table and identify the minimum-cost tour.

MODULE 3 — Backtracking, State-Space Trees & Lower Bounds

Scope: Backtracking principle and terminology; live, E, success and dead nodes; explicit and implicit constraints; state-space trees; N-Queens/4-Queens; Sum of Subsets; Branch and Bound/8-puzzle; lower-bound theory and comparison trees.

> **Previous-paper signal:** The uploaded October 2025 paper includes lower-bound theory with an example (4 marks), and 8-puzzle using Branch and Bound or state-space trees in backtracking using 4-Queens (8 marks).

A. 4-MARK QUESTIONS

Write each as a complete short answer, aiming for about 2 handwritten pages as requested. Include a definition, key points and a small example/diagram where appropriate.

[VERY HIGH] 1. Define lower-bound theory and give an example.

[VERY HIGH] 2. Define backtracking and explain when it is used.

[HIGH] 3. Explain live node, E-node, success node and dead node.

[HIGH] 4. Differentiate explicit and implicit constraints in backtracking.

[HIGH] 5. What is a state-space tree? Explain its role in backtracking.

[HIGH] 6. State the 4-Queens problem and its constraints.

[MEDIUM] 7. Explain the Sum of Subsets problem.

[MEDIUM] 8. What is Branch and Bound? Explain its basic idea as covered in the notes.

[MEDIUM] 9. Explain the role of pruning in backtracking.

[MEDIUM] 10. What is a comparison tree?

B. 8-MARK QUESTIONS

Aim for 3–4 handwritten pages. Algorithm questions should include steps/pseudocode, a trace or diagram, and complexity where the notes support it.

[VERY HIGH] 1. Explain how state-space trees are built in backtracking, using the 4-Queens problem as an example.

[VERY HIGH] 2. Demonstrate solving the 8-puzzle using Branch and Bound, as specified in the previous paper.

[HIGH] 3. Explain the backtracking algorithm step by step, including choice, validity checking and backtracking.

[HIGH] 4. Explain the Sum of Subsets problem using a state-space tree, identifying promising and non-promising branches.

[HIGH] 5. Explain lower-bound theory using comparison trees for sorting or searching, following the module notes.

[MEDIUM] 6. Explain the terms and general working of backtracking with a diagram.

[MEDIUM] 7. Compare backtracking and Branch and Bound as presented in your notes.

[MEDIUM] 8. Explain explicit and implicit constraints and show how they restrict the solution space.

C. WORKED-PROBLEM / DIAGRAM PRACTICE

1. Draw a state-space tree for the 4-Queens problem and mark valid and dead branches.

2. Trace a Sum of Subsets example for a given set and target sum.

3. Practise the 8-puzzle example used in your notes, showing each state and the rule used to choose the next state.

4. Explain a lower-bound argument using a comparison tree from the notes.

MODULE 4 — Complexity Theory & Network Flows

Scope: Complexity theory; Class P and NP; polynomial-time reductions; NP-Hard and NP-Complete; Vertex Cover and Clique; flow networks; network flow; Max-Flow Min-Cut theorem; Ford–Fulkerson method.

> **Previous-paper signal:** The uploaded October 2025 paper includes flow-network properties (4 marks) and P vs NP or Max-Flow Min-Cut with a network diagram (8 marks).

A. 4-MARK QUESTIONS

Write each as a complete short answer, aiming for about 2 handwritten pages as requested. Include a definition, key points and a small example/diagram where appropriate.

[VERY HIGH] 1. Explain the properties of a flow network.

[VERY HIGH] 2. Define Class P and Class NP.

[HIGH] 3. Differentiate NP-Hard and NP-Complete problems.

[HIGH] 4. What is a decision problem? Give an example.

[HIGH] 5. Define polynomial-time reduction and state its purpose.

[HIGH] 6. Define Vertex Cover and Clique.

[HIGH] 7. What is a residual network or augmenting path in a flow network?

[MEDIUM] 8. State the Max-Flow Min-Cut theorem.

[MEDIUM] 9. Explain flow conservation and capacity constraints.

[MEDIUM] 10. Why is the question P = NP important, according to the notes?

B. 8-MARK QUESTIONS

Aim for 3–4 handwritten pages. Algorithm questions should include steps/pseudocode, a trace or diagram, and complexity where the notes support it.

[VERY HIGH] 1. Differentiate Class P and Class NP with respect to verifiability versus solvability.

[VERY HIGH] 2. Prove or explain the Max-Flow Min-Cut theorem with an example network diagram.

[HIGH] 3. Explain Ford–Fulkerson method using augmenting paths and show the flow updates for a network.

[HIGH] 4. Explain polynomial-time reductions and their role in establishing NP-Hardness or NP-Completeness.

[HIGH] 5. Explain Class P, Class NP, NP-Hard and NP-Complete and show their relationship.

[HIGH] 6. Explain the Vertex Cover problem and the Clique problem as examples in complexity theory.

[MEDIUM] 7. Explain the features and purpose of complexity theory.

[MEDIUM] 8. Explain flow networks, source, sink, capacity, flow conservation and cut capacity.

C. WORKED-PROBLEM / DIAGRAM PRACTICE

1. Given a flow network, identify source and sink, capacities and flow-conservation constraints.

2. Trace Ford–Fulkerson: find an augmenting path, determine bottleneck capacity, update residual capacities and repeat.

3. Practise a small Max-Flow Min-Cut example and compare maximum flow with minimum cut capacity.

MODULE 5 — Approximation & Randomized Algorithms

Scope: Approximation algorithms; approximation ratio; 2-approximation algorithm for Vertex Cover; randomized algorithms; basic probability; polynomial identity testing; randomized Quick Sort; practical exposure to the Schwartz–Zippel Lemma.

> **Previous-paper signal:** The uploaded October 2025 paper includes significance and uses of randomized algorithms (4 marks), and merits/demerits of randomized versus deterministic algorithms (8 marks).

A. 4-MARK QUESTIONS

Write each as a complete short answer, aiming for about 2 handwritten pages as requested. Include a definition, key points and a small example/diagram where appropriate.

[VERY HIGH] 1. Explain the significance of randomized algorithms and mention their uses.

[VERY HIGH] 2. Define an approximation algorithm and approximation ratio.

[HIGH] 3. Differentiate an exact algorithm and an approximation algorithm.

[HIGH] 4. What is a randomized algorithm? How can it differ from a deterministic algorithm?

[HIGH] 5. State the main steps of the 2-approximation algorithm for Vertex Cover.

[HIGH] 6. Define random experiment, sample space, event and probability.

[MEDIUM] 7. Explain the Las Vegas and Monte Carlo types of randomized algorithms as shown in the notes.

[MEDIUM] 8. What is polynomial identity testing?

[MEDIUM] 9. What is the purpose of the Schwartz–Zippel Lemma in polynomial identity testing?

[MEDIUM] 10. Explain randomized Quick Sort in brief.

B. 8-MARK QUESTIONS

Aim for 3–4 handwritten pages. Algorithm questions should include steps/pseudocode, a trace or diagram, and complexity where the notes support it.

[VERY HIGH] 1. Compare and contrast the merits and demerits of randomized algorithms over deterministic algorithms in large-scale computation.

[VERY HIGH] 2. Explain the 2-approximation algorithm for Vertex Cover step by step, with an example and its approximation guarantee.

[HIGH] 3. Define approximation ratio and explain it for minimization and maximization problems with examples.

[HIGH] 4. Explain randomized Quick Sort, including random pivot selection and its effect on behaviour.

[HIGH] 5. Explain polynomial identity testing and the practical role of the Schwartz–Zippel Lemma, following the module notes.

[HIGH] 6. Review basic probability rules relevant to randomized algorithms and illustrate them with examples.

[MEDIUM] 7. Compare Las Vegas and Monte Carlo algorithms in terms of correctness and running time, using the notes.

[MEDIUM] 8. Explain why approximation algorithms are useful for difficult optimization problems.

C. WORKED-PROBLEM / DIAGRAM PRACTICE

1. Run the 2-approximation Vertex Cover algorithm on a small graph: choose an edge, add both endpoints, remove incident edges, and repeat.
2. Calculate the approximation ratio for a provided approximate solution and optimal solution.
3. Practise probability calculations using the formulas and examples shown in the module notes.

FINAL REVISION CHECKLIST

Tick each item only when you can explain it without reading your notes.

Module	Before the exam, make sure you can…
Module 1	Explain O, Ω and Θ; trace Merge Sort and Quick Sort; solve a recurrence using substitution or iteration.
Module 2	Explain greedy control abstraction; construct an MST; solve a fractional knapsack or TSP-DP example.
Module 3	Draw a 4-Queens state-space tree; explain backtracking terms; practise lower-bound and 8-puzzle questions from the notes.
Module 4	Differentiate P and NP; define NP-Hard/NP-Complete; trace Ford–Fulkerson; explain Max-Flow Min-Cut.
Module 5	Explain approximation ratio; run the 2-approximation Vertex Cover algorithm; compare randomized and deterministic algorithms.

Exam strategy: because the uploaded paper requires Part B coverage across modules, prepare at least one confident 8-mark answer from each module. Also revise the corresponding 4-mark definitions and distinctions.

Source note: DAA Module 1, Module 2, Module 3, Module 4, Module 5 and the uploaded “S3 previous year question papers” PDF. Module 5 contains visual pages, so its question bank follows the readable topic headings and diagrams. Where a question requires a numerical example, use the specific worked examples in your class notes.